COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate course for games

09

Lab and problem workbook

20 practical routes · 40 written problems · 25 review questions

PACKAGE 09 / VOLUME 08 / SEPTEMBER 28, 2026
Modules 36–40 · Gate H · Expanded teaching edition 1.0
Study, implement, test and review. Models are not physical measurements.

Contents

What counts as completed practical work 3

Entry diagnostic 3

Module 36 — Practical work 4

Lab 36.1 — Serial interfaces and peripheral contracts 5

Lab 36.2 — Firmware, RTOS scheduling and driver lifecycle 7

Lab 36.3 — Schematics, PCB layout, power integrity and signal integrity 9

Lab 36.4 — Bring-up, instrumentation and fault isolation 11

Module 36 — Project and review 13

Module 37 — Practical work 14

Lab 37.1 — SIMT execution, divergence, occupancy and latency hiding 15

Lab 37.2 — Coalescing, local storage, caches and shared-memory banks 17

Lab 37.3 — Raster, texture, matrix and ray-tracing units 19

Lab 37.4 — CPU-GPU sharing, discrete memory and synchronization 21

Module 37 — Project and review 23

Module 38 — Practical work 24

Lab 38.1 — Amdahl, roofline and bottleneck models 25

Lab 38.2 — Profilers, counters and trustworthy benchmark design 27

Lab 38.3 — SIMD, cache blocking, NUMA and job scheduling 29

Lab 38.4 — Energy, thermal behavior and sustained performance 31

Module 38 — Project and review 33

Module 39 — Practical work 34

Lab 39.1 — Input-to-display latency and frame pacing 35

Lab 39.2 — Mobile and handheld systems: memory, power and lifecycle 37

Lab 39.3 — XR spaces, stereo views, pose timing and comfort constraints 39

Lab 39.4 — Portability, platform services and console-development boundaries 41

Module 39 — Project and review 43

Module 40 — Practical work 44

Lab 40.1 — Workload characterization and measurable requirements 45

Lab 40.2 — Architecture alternatives, arithmetic precision and dataflow 47

Lab 40.3 — Interface design, virtualization and end-to-end evaluation 49

Lab 40.4 — Design review, evidence tiers and undergraduate CE completion 51

Module 40 — Project and review 53

Reading directory 53

What counts as completed practical work

Each lesson has an original practical assignment plus a smaller automatically checked computational slice. Complete both. The slice supplies a testable mechanism; it does not silently replace a schematic/layout, engine optimization, strict backend or integrated capstone. Full pass criteria and evidence are reproduced below. Reference solutions are supplied for the slices; design work is assessed using the stated evidence and rubric because there is no single correct board, profile or architecture.

Implement `solve(case)` in the matching file under labs/starters. Read its contract and the JSON cases. Do not alter tests merely to make an incorrect output pass. Use `python -m labs.run check 36.1 --student`, replacing the lesson ID. All reference cases can be checked with `python -m labs.run all`; the broader suite is `python -m unittest discover -s labs/tests -v`.

Record predictions before running the demonstration, then explain disagreements. Save raw evidence and metadata, answer the two written problems per lesson and attempt the transfer task without copying the reference. The transfer tasks are uncounted extensions; the package contains exactly 40 counted written problems and 25 separate review questions.

Entry diagnostic

These ten questions are a study aid, not a validated placement exam. Repair gaps before claiming module completion; a peer should review unresolved memory-ownership, timing or electrical-safety misunderstandings.

D01. A 32-bit word starts at address 62 and a modeled sector is 64 bytes. How many sectors can it touch?

D02. Does a volatile C++ pointer establish CPU/device ownership or API-specific synchronization?

D03. A 10 ms frame contains a 2 ms serial operation. What is the maximum additive-frame speedup if it disappears?

D04. Explain why deadline and average utilization are different concepts.

D05. What is the difference between a functional model and a physically measured circuit?

D06. A camera pose has rotation R and position p. Give the rigid world-to-camera inverse.

D07. What evidence should accompany a claimed faster algorithm?

D08. Why can a timeout fail to make a device buffer safe to reuse?

D09. Convert 3 MiB and 3 MB to bytes.

D10. What must happen before optional physical board bring-up?

Module 36 — Practical work

Prerequisites: Modules 17-18 and 31-35. Physical work remains optional and supervised.

Original project: Design a simulated controller or peripheral with firmware, a host interface and a schematic-level review. A physical board extension begins only after the electrical, layout and bring-up plans receive qualified review.

Expanded project plan

Build a small controller pipeline: a modeled peripheral supplies signed input samples; a bounded firmware scheduler processes them; a host adapter consumes only valid current data. Keep the provided packet format or document a replacement completely. The scheduler slice and packet slice must be connected by explicit request timing, not just run as unrelated demos.

Create a low-speed schematic and a simple or unrouted PCB layout in KiCad, as required by the original lesson. Include exact selected part/connector footprints, a power-domain table and a return-path review. Fabrication, soldering and physical power-up are not required. The JSON linter is only a review checklist and does not replace KiCad files or ERC/DRC evidence.

Inject wrong mode metadata, bad checksum, stale sequence, timeout, overloaded processing, removal during a request and a simulated missing clock. Demonstrate a bounded safe response for each. Where real SPI cannot diagnose a mode mismatch from bytes alone, state the stronger information available in your simulator.

Deliver source, interface contract, timing trace, lifecycle/cancellation policy, schematic/layout files, ERC/DRC report or justified findings, and a bring-up hypothesis table. Acceptance requires no silent stale-input acceptance, no unbounded retries and no invalid buffer lifetime after removal. Carry the ownership contract into Gate H.

Lab 36.1 — Serial interfaces and peripheral contracts

Original practical requirement

Build a simulated peripheral transaction decoder and driver for one interface. Pass: incorrect framing or mode is detected, timeouts are bounded and a complete interface-contract table is provided.

Automatically checked slice

Decode a five-byte educational SPI packet. Fields: mode, expected_mode, elapsed_us, timeout_us, expected_seq, frame. Modes 0..3, bytes 0..255. Packet [0xA5, seq, signed16_hi, signed16_lo, XOR of preceding bytes]. Deadline equality succeeds. The simulator knows endpoint mode metadata; real SPI mode faults are not always self-identifying.

Starter: `labs/starters/lesson_36_1.py`

Demo: `python -m labs.run demo 36.1`

Student check: `python -m labs.run check 36.1 --student`

Published contract cases: 9. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "mode": 0,
  "expected_mode": 0,
  "elapsed_us": 40,
  "timeout_us": 50,
  "expected_seq": 0,
  "frame": [165, 0, 0, 5, 160]
}
```

Full lab procedure

1. Copy the interface-contract template and choose exactly one modeled command. Specify byte order, signedness, clock mode, timing origin, sequence policy and recovery.
2. Predict the valid packet and each error status by hand. Run `python -m labs.run demo 36.1`, then implement the starter decoder without importing the reference.
3. Add tests for truncated frames, corrupt checksums, stale sequences, signed endpoints, mode mismatch and a completion exactly at the deadline. Explain why real mode detection is weaker.
4. Connect the decoder to a small host-request state machine. Inject a timeout followed by a late reply; demonstrate that the old result is not accepted as the next input sample.

Evidence and acceptance

Keep the interface table, at least six transaction traces, passing starter checks, a timeout/late-response trace and the limitation statement. The host state machine is learner work beyond the five-byte decoder.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P36.1-A. For UART 8-N-1 at 115200 bit/s, derive the minimum wire time for a 12-byte payload, ignoring interbyte gaps.

P36.1-B. A device allows at most 100 microseconds from request start to valid reply. Wire time is 50, processing is bounded at 20, and host jitter is bounded at 18 microseconds. What margin remains?

Transfer task

Change the payload to two axes and a button bitmask. Keep the protocol version explicit and prove the old decoder rejects the incompatible format instead of silently misreading it.

Assigned study: R53: device drivers, UART, SPI and I2C APIs; R51: bus-level driver documentation. For hardware, add the exact peripheral datasheet.

Lab 36.2 — Firmware, RTOS scheduling and driver lifecycle

Original practical requirement

Use a supported RTOS simulator or a deterministic scheduler model to implement input, processing and logging tasks. Pass: deadlines are evaluated under injected load and removal/cancellation leaves no live invalid requests.

Automatically checked slice

Simulate integer-tick, single-core, preemptive fixed-priority periodic tasks. Each task has name, period, cost, deadline and priority (lower wins). All first releases occur at tick 0. horizon is 1..10000; deadlines are relative and <= period. Deadline misses count unfinished jobs when their deadline occurs, not merely at horizon.

Starter: `labs/starters/lesson_36_2.py`

Demo: `python -m labs.run demo 36.2`

Student check: `python -m labs.run check 36.2 --student`

Published contract cases: 7. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "horizon": 10,
  "tasks": [
    {
      "name": "input",
      "period": 5,
      "cost": 1,
      "deadline": 5,
      "priority": 0
    }
  ]
}
```

Full lab procedure

1. Specify input, processing and logging tasks with integer execution budgets, periods, deadlines and priorities. Draw a short schedule before running the model.
2. Implement the starter and compare normal load, a long logger and a burst of critical work. Record release, completion, miss and pending counts; do not discard late jobs from the report.
3. Create a second driver lifecycle test with pending requests at suspend, cancellation and removal. Reuse the Lesson 40.3 reference only as a study example, not as the learner solution.
4. Explain a priority-inversion trace and redesign the critical path to avoid a logging dependency. List the timing costs omitted from your scheduler and how a physical extension would measure them.

Evidence and acceptance

Submit a task table, predicted and observed schedules, overload evidence, cancellation/removal traces and an explicit list of unsupported RTOS behaviors. A no-miss finite trace is not a universal deadline proof.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P36.2-A. Compute utilization for C/T values 1/5, 2/10 and 3/20. Does the sum establish deadlines?

P36.2-B. A deadline is 5 ms. The task needs 1 ms of execution, waits 3 ms on a lock, and suffers 2 ms of higher-priority interference. Can that response meet the deadline under an additive model?

Transfer task

Add bounded release jitter to your own scheduler and search for a deadline-miss trace. Explain the distinction between a found counterexample and a complete schedulability proof.

Assigned study: R53: kernel scheduling, synchronization and device model; R51: driver lifecycle and power management.

Lab 36.3 — Schematics, PCB layout, power integrity and signal integrity

Original practical requirement

Create a low-speed peripheral schematic and unrouted or simple two-layer layout in KiCad. Pass: footprints are checked against datasheets, ERC/DRC findings are resolved or justified and a reviewer inspects power and return paths before fabrication.

Automatically checked slice

Lint a deliberately small board-review record, not a KiCad file. Required fields: supply_v, min_v, max_v, load_ma, budget_ma, footprint_verified and return_path_continuous. Report sorted issues. Even an empty list means only ready for qualified review, never authorization to fabricate or power hardware.

Starter: `labs/starters/lesson_36_3.py`

Demo: `python -m labs.run demo 36.3`

Student check: `python -m labs.run check 36.3 --student`

Published contract cases: 7. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "supply_v": 3.3,
  "min_v": 3.0,
  "max_v": 3.6,
  "load_ma": 40,
  "budget_ma": 100,
  "footprint_verified": true,
  "return_path_continuous": true
}
```

Full lab procedure

1. Create a new KiCad project for one low-speed peripheral. Record the installed version and select exact symbols/footprints with datasheet references.
2. Draft the schematic, connector orientation table and power budget. Run ERC; resolve or explicitly justify each finding.
3. Place components on an unrouted or simple two-layer board. Annotate supply and return paths, run DRC and preserve the report even when findings remain for an intentionally unrouted design.
4. Implement the JSON review checker as a separate exercise. Present both artifacts for qualified review; do not fabricate or energize a board on the strength of the checker.

Evidence and acceptance

Provide native KiCad project files, schematic export, layout screenshots or PDF, checked pin/footprint table, ERC/DRC reports and a review status. An unrouted layout must be labeled unrouted, not manufacturing-ready.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P36.3-A. An ideal 25 mA transient lasts 4 microseconds. What capacitance limits ideal droop to 50 mV?

P36.3-B. A connector drawing labels pin 1 on the left when viewed from the mating side. A footprint screenshot labels pin 1 on the right from the component side. Is one necessarily wrong?

Transfer task

Choose one unresolved ERC/DRC item and write a proper waiver candidate with rule, reason, evidence, reviewer and expiry condition. A reviewer may reject the waiver.

Assigned study: R36: schematic editor, PCB editor, ERC and DRC; R30: nonideal circuit behavior; R49: interconnect and packaging extension.

Lab 36.4 — Bring-up, instrumentation and fault isolation

Original practical requirement

Write and rehearse a bring-up procedure against a simulated device with injected clock, reset and protocol faults. Pass: each fault has a safe diagnostic branch and the procedure includes explicit stop conditions.

Automatically checked slice

Choose the first diagnostic action from Boolean observations: over_limit, power_ok, reset_released, clock_ok, id_ok, framing_ok. Check safety first, then power, reset, clock, identity and framing. This is a simulated decision tree, not a device-specific physical procedure.

Starter: `labs/starters/lesson_36_4.py`

Demo: `python -m labs.run demo 36.4`

Student check: `python -m labs.run check 36.4 --student`

Published contract cases: 7. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "over_limit": false,
  "power_ok": true,
  "reset_released": true,
  "clock_ok": true,
  "id_ok": true,
  "framing_ok": true
}
```

Full lab procedure

1. Write a decision table for power, reset, clock, identity and transaction checks. Include an over-limit stop and an unknown-observation branch.
2. Implement the starter decision function and run each single-fault case. Inject two simultaneous faults and predict which dependency should dominate.
3. Rehearse three scripted sessions: clock absent, reset held and protocol framing incorrect. Retain the sequence of hypotheses rather than only the final status.
4. Write an instrument-selection note for a hypothetical physical extension. Name required manuals and review prerequisites; do not claim that the Python model validates an electrical waveform.

Evidence and acceptance

Submit the procedure, expected-observation table, single- and multiple-fault traces, explicit stop/restart conditions, and one incident note separating established causes from untested hypotheses.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P36.4-A. Two hypotheses are wrong bus mode and missing clock. The decoder reports zeros for every reply. What extra observation discriminates better than repeating the read?

P36.4-B. A simulated setup exceeds its declared current budget and also reports an incorrect device ID. Which result has priority?

Transfer task

Have another learner inject an undisclosed fault. Run your procedure without editing it mid-session, then revise only the parts whose observations failed to distinguish hypotheses.

Assigned study: R53: board support and debugging; R36: fabrication outputs and design review; selected instrument manuals for any physical extension.

Module 36 — Project and review

A simulated controller with a reviewable board plan

Use the expanded project plan at this module's opening. Bring its implementation and evidence to the review below.

Project review rule

Assess contract/requirements, implementation, verification, evidence/reproducibility and limitations at 0–4 points each. A suggested readiness target is 16/20 after revision, with no unresolved correctness, ownership or safety defect. This local study rubric is not an accredited assessment. Gate H has its own integrated rubric.

Five-question module review

R36-1. A controller returns a checksum-valid packet with an old sequence after a retry. Explain what additional contract is needed.

R36-2. An input task misses its deadline while average CPU load is 40%. Give a plausible structural cause and an experiment.

R36-3. A board passes DRC but a connector footprint was mirrored. Explain why DRC may not catch the error.

R36-4. Power is within the reviewed limit, reset is released, clock is absent and identity reads fail. Which hypothesis should be tested first?

R36-5. Define evidence sufficient for the simulated peripheral project and evidence still needed for fabrication.

Have a reviewer select one adverse case and rerun it from your instructions. Record the reviewer's concern, evidence examined and correction or reasoned response.

Module 37 — Practical work

Prerequisites: Modules 16, 22, 32 and 34.

Original project: Create a GPU architecture workbook connecting shader behavior to execution groups, registers, memory transactions and fixed-function resources. Measure one accessible GPU or clearly label a simulator-only study.

Expanded project plan

Select a simple transform, image filter or particle operation with a software oracle. Map its control flow, logical execution groups, live values, memory addresses and fixed-function opportunities. Work through at least one branch-divergent case, one partial group, one aligned and one strided access pattern, and one shared-bank/broadcast example.

Compare two layouts with conversion included and two CPU/device data-placement strategies with clear ownership. State group width, sector size, bank mapping, rates and precision as assumptions unless verified for a named target. An accessible GPU measurement is welcome but not required; a simulator-only workbook is acceptable when clearly labeled.

Deliver calculations, executable slices, expected-result checks, a resource-service table, an end-to-end budget and a page explaining which profiler observations would confirm or reject your dominant-bottleneck hypothesis. The workbook must not equate occupancy with speed, modeled sectors with measured DRAM bytes, or shared addressing with free synchronization.

Lab 37.1 — SIMT execution, divergence, occupancy and latency hiding

Original practical requirement

Compare coherent and divergent workloads, varying register pressure or group size where supported. Pass: outputs agree, device group assumptions are recorded and the report does not equate occupancy with performance.

Automatically checked slice

Compute active-lane efficiency in an educational serialized-path group. paths is a nonempty list of A/B values of length <= width. Path costs and common cost are integer issue slots; at least one slot must execute. Count inactive tail lanes in capacity. This is not a cycle-accurate GPU model or an occupancy calculator.

Starter: `labs/starters/lesson_37_1.py`

Demo: `python -m labs.run demo 37.1`

Student check: `python -m labs.run check 37.1 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "width": 4,
  "paths": ["A", "A", "A", "A"],
  "common": 0,
  "a_cost": 4,
  "b_cost": 4
}
```

Full lab procedure

1. Implement the lane-efficiency model and verify coherent, split, unequal-path and tail cases by hand.
2. Build a hypothetical residency table varying registers per thread and workgroup size. State which hardware constraints and allocation granularity are omitted.
3. On an available GPU, optionally compare equivalent workloads with recorded device properties; otherwise compare clearly labeled model workloads and do not invent GPU times.
4. Test a grouping or sorting strategy. Include rearrangement and restoration costs in a separate application estimate, and explain why occupancy alone cannot select the winner.

Evidence and acceptance

Provide output equivalence, input-distribution description, group assumptions, issue-slot calculations and either measured device evidence or an explicit simulator-only label.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P37.1-A. A hypothetical register file has 32768 registers. Each 128-thread block uses 64 registers per thread. Ignoring every other limit, how many blocks fit?

P37.1-B. Three of four lanes use a one-slot path; one uses a nine-slot path. With no common prefix, compute useful lane-slot efficiency.

Transfer task

Create two distributions with identical branch probabilities but different grouping coherence. Explain why global branch frequency does not determine per-group divergence.

Assigned study: R37: programming model, execution and hardware implementation; R48: vector and multithreaded architecture foundations.

Lab 37.2 — Coalescing, local storage, caches and shared-memory banks

Original practical requirement

Benchmark contiguous, strided and tiled access with identical useful work. Pass: count useful bytes, state the relevant memory level and include any layout-conversion cost in the application comparison.

Automatically checked slice

Count unique byte-addressed sectors touched by one modeled memory instruction, including cross-sector words and repeated addresses. Fields: addresses, word_bytes (1..128), sector_bytes (1..4096). useful_unique_bytes is not the logical byte count for broadcast reads. Cache reuse, merging across instructions and actual DRAM traffic are not simulated.

Starter: `labs/starters/lesson_37_2.py`

Demo: `python -m labs.run demo 37.2`

Student check: `python -m labs.run check 37.2 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "addresses": [0, 4, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44, 48, 52, 56, 60, 64, 68, 72,
76, 80, 84, 88, 92, 96, 100, 104, 108, 112, 116, 120, 124],
  "word_bytes": 4,
  "sector_bytes": 32
}
```

Full lab procedure

1. Implement the sector counter, including cross-sector words and broadcast reads. Predict every supplied case before running it.
2. Add a bank-index worksheet for row widths 32 and 33, recording the unique words per bank rather than just counting lanes.
3. Compare contiguous, strided and tiled access for identical useful work. Use measured GPU data when available; otherwise present model counts and avoid invented hardware bandwidth.
4. Include layout-conversion and synchronization costs in the full comparison. Exercise a partial tile, a nonmultiple size and a repeated-address case.

Evidence and acceptance

Retain address traces, useful/logical/transferred-byte definitions, bank calculations, output validation and an amortization calculation for the representation change.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P37.2-A. Eight four-byte words start at addresses 4,8,...,32 with 32-byte sectors. How many sectors and useful bytes are involved?

P37.2-B. A read group requests word addresses 0,32,64,96 in a 32-bank model. Compare with addresses 0,0,0,0.

Transfer task

Introduce a small cache into your own address model. Show a trace where sector counts per instruction remain high but external-memory traffic falls through reuse.

Assigned study: R37: device memory access, shared memory and performance considerations; R43: operational intensity.

Lab 37.3 — Raster, texture, matrix and ray-tracing units

Original practical requirement

Analyze one texture-heavy, arithmetic-heavy or ray-query workload. Pass: identify the likely resource limits using measurements or a documented model and state supported precision and feature assumptions.

Automatically checked slice

Compute a resource-service lower bound with ideal overlap and a serial service estimate. demands and rates are dictionaries with identical nonempty resource keys; setup_s is nonnegative. All rates must be positive. Neither result is a hardware timing guarantee: queuing and dependencies are omitted.

Starter: `labs/starters/lesson_37_3.py`

Demo: `python -m labs.run demo 37.3`

Student check: `python -m labs.run check 37.3 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "demands": {
    "alu": 100,
    "tex": 20
  },
  "rates": {
    "alu": 100,
    "tex": 100
  },
  "setup_s": 0.1
}
```

Full lab procedure

1. Choose one workload class and define identical useful work and an explicit numerical or visual-quality target.
2. Build a resource-demand table and implement the lower-bound calculator. Predict which demand dominates under each rate assumption.
3. Sweep at least three batch sizes, including one below the predicted crossover. Include setup, conversion and result-consumption costs.
4. Validate outputs or the declared quality proxy and record supported features. Label model-only results clearly; a resource-rate assumption is not a measured counter.

Evidence and acceptance

Submit the demand/rate table, numerical-quality checks, batch crossover derivation, phase-cost comparison and the evidence supporting the proposed bottleneck.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P37.3-A. Arithmetic service is 3 ms, ray service is 5 ms and setup is 0.4 ms. Give the ideal-overlap lower estimate and serial-service comparator.

P37.3-B. A specialized path has lower mean image error but creates one severe outlier in 1000 frames. What should the comparison report?

Transfer task

Change the quality tolerance and repeat the architecture decision. Identify whether the performance winner changes because the supported numerical format changes.

Assigned study: R37: architecture and specialized operations; R21: graphics/compute capabilities; R23: ray intersection and acceleration reference.

Lab 37.4 — CPU-GPU sharing, discrete memory and synchronization

Original practical requirement

Compare an upload/compute/readback path with an available shared-memory path, or model both with stated assumptions. Pass: correct ownership, separate phase timings and no claim that unified means zero-cost.

Automatically checked slice

Model one serialized request with setup, upload, compute, readback and synchronization. input_bytes/output_bytes count payload; bandwidth_Bps is effective payload bytes/second. For shared=True, use visibility_s instead of explicit copies. compute_s already includes kernel memory service. Only explicit transfer phases are removed; ownership and visibility costs remain.

Starter: `labs/starters/lesson_37_4.py`

Demo: `python -m labs.run demo 37.4`

Student check: `python -m labs.run check 37.4 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "input_bytes": 1000,
  "output_bytes": 1000,
  "bandwidth_Bps": 1000,
  "setup_s": 0.1,
  "compute_s": 0.2,
  "sync_s": 0.1,
  "visibility_s": 0.3,
  "shared": false
}
```

Full lab procedure

1. Draw an ownership table for one request and for two overlapping buffers. Specify exactly when writes, reads and reuse are permitted.
2. Implement the staged-cost calculator with consistent units. Compare an explicit-copy path with a stated shared-visibility path.
3. Measure phase timing on an accessible backend or model it with declared assumptions. Keep CPU and GPU clock domains separate unless an API supports calibration.
4. Inject a delayed completion and a timeout; demonstrate why premature buffer reuse is prohibited. Compare a CPU-consumed result with a device-resident result.

Evidence and acceptance

Provide the ownership state machine, phase timings or estimates, memory-lifetime assumptions, timeout trace and an end-to-end comparison with the same useful result boundary.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P37.4-A. Upload is 0.3 ms, compute 0.8 ms and readback 0.2 ms. Ignoring all other work, compare serialized one-request latency with the ideal steady interval of three independent stages.

P37.4-B. A buffer has been submitted but the host wait times out. Name two conditions that could safely enable eventual reuse.

Transfer task

Add a third request to the double-buffer schedule and identify the first point at which the host must wait or allocate another buffer. Explain the memory-versus-latency tradeoff.

Assigned study: R37: memory management and synchronization; R21, R24 or R25: the selected platform's resource-sharing contract.

Module 37 — Project and review

An architecture workbook tied to one shader or kernel

Use the expanded project plan at this module's opening. Bring its implementation and evidence to the review below.

Project review rule

Assess contract/requirements, implementation, verification, evidence/reproducibility and limitations at 0–4 points each. A suggested readiness target is 16/20 after revision, with no unresolved correctness, ownership or safety defect. This local study rubric is not an accredited assessment. Gate H has its own integrated rubric.

Five-question module review

R37-1. A shader changes from 100% to 75% theoretical occupancy and becomes faster. Is this contradictory?

R37-2. A layout reduces modeled sectors by half but adds a conversion pass. State the correct comparison.

R37-3. A matrix unit has high nominal arithmetic throughput. Why might a small transform batch lose?

R37-4. Shared addressing is available. Which other questions remain before CPU/GPU overlap is correct?

R37-5. Explain how to use a GPU-less architecture workbook honestly.

Have a reviewer select one adverse case and rerun it from your instructions. Record the reviewer's concern, evidence examined and correction or reasoned response.

Module 38 — Practical work

Prerequisites: Modules 09, 16, 18, 30 and 37.

Original project: Produce a performance investigation with a hypothesis, controlled workload, raw measurements, uncertainty, a bottleneck model and a verified optimization. A negative result is acceptable when the evidence is rigorous.

Expanded project plan

Choose a particle, transform or other engine workload and establish a suitable software baseline. Predeclare input sizes, quality, warmup, repetition, comparison order and exclusion rules. Add an intentional no-op or wrong-result path and verify that the harness rejects it before reporting performance.

Apply one locality change and one scheduling change, testing them separately before their combination. Measure small, tail and large batches. Record raw samples and metadata, report central and tail behavior, and use an Amdahl or roofline worksheet to bound the application opportunity. A counter observation supports a hypothesis only with an explanation and a controlled test.

Analyze the supplied synthetic energy trace or collect a normal-limits sustained trace with its measurement boundary. Deliver the protocol, source, raw timings, correctness failures, numerical-error results, model, energy analysis and a reasoned conclusion. An honest negative result meets the project objective when the experiment is sufficiently controlled and reproducible.

Lab 38.1 — Amdahl, roofline and bottleneck models

Original practical requirement

Build a roofline worksheet for one kernel and an Amdahl estimate for one frame optimization. Pass: units cancel correctly, modeled ceilings use the declared memory level and observed deviations are explained as hypotheses.

Automatically checked slice

Return an Amdahl speedup and roofline bound. Inputs: unchanged_fraction in [0,1], acceleration >0, added_fraction >=0, compute_GFLOPs >0, bandwidth_GBs >0, intensity_FLOP_per_byte >=0. Added overhead is expressed as a fraction of ORIGINAL runtime. Decimal GB and GFLOP units cancel.

Starter: `labs/starters/lesson_38_1.py`

Demo: `python -m labs.run demo 38.1`

Student check: `python -m labs.run check 38.1 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "unchanged_fraction": 0.2,
  "acceleration": 4,
  "added_fraction": 0,
  "compute_GFLOPs": 1000,
  "bandwidth_GBs": 100,
  "intensity_FLOP_per_byte": 2
}
```

Full lab procedure

1. Reproduce the original roofline and Amdahl calculations by hand, including units and memory level.
2. Implement the supplied calculator and test zero intensity, entirely unchanged work, and overhead that makes an optimization slower.
3. Select an engine kernel; measure or explicitly estimate its traffic and useful operations. Build a parameter table identifying measured, documented and assumed values.
4. Compare at least three batch sizes or layouts with the model. Treat unexplained gaps as hypotheses, not automatic hardware diagnoses.

Evidence and acceptance

Submit a dimensionally checked worksheet, an application critical-path diagram, raw timing or clearly labeled estimates, and a sensitivity table covering optimistic and adverse bandwidth/overhead values.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P38.1-A. A frame has unchanged fraction 0.3; the remainder improves 5x and new overhead is 0.06 of original time. Find speedup.

P38.1-B. Compute ceiling is 600 GFLOP/s and DRAM bandwidth is 75 GB/s. At 4 FLOP/byte, find the roof and ridge.

Transfer task

Redraw your frame model with the candidate operation overlapping an unrelated task. Explain when removing its duration stops shortening the critical path.

Assigned study: R43: original Roofline model; R48: quantitative architecture; revisit Module 18 parallel limits.

Lab 38.2 — Profilers, counters and trustworthy benchmark design

Original practical requirement

Create a benchmark harness with warmup, repeated runs, output validation and metadata. Pass: an intentional no-op or wrong-result variant is rejected and all included/excluded samples are documented.

Automatically checked slice

Summarize raw times only after every output checksum passes. Fields: times_ns, checksums, expected_checksum, warmup (number of leading samples excluded). Positive finite times required; all samples, including warmup, must validate. Report retained n, mean, median and nearest-rank p95; no post-hoc outlier removal.

Starter: `labs/starters/lesson_38_2.py`

Demo: `python -m labs.run demo 38.2`

Student check: `python -m labs.run check 38.2 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "times_ns": [100, 10, 20, 30],
  "checksums": [7, 7, 7, 7],
  "expected_checksum": 7,
  "warmup": 1
}
```

Full lab procedure

1. Write a benchmark protocol before running code: exact useful result, timing boundaries, warmup, repetitions, comparison order and exclusions.
2. Implement the sample analyzer and reject a wrong checksum even when it occurs during warmup. Verify the nearest-rank percentile by hand.
3. Build a harness for your engine operation; add intentionally wrong-output and no-op variants and demonstrate their rejection.
4. Collect raw samples and metadata, then report central tendency, tail statistic, sample count and a reasoned uncertainty statement. Re-run in a fresh session.

Evidence and acceptance

Provide the predeclared protocol, raw data, environment record, wrong-result rejection log, inclusive and any filtered summaries, and a conclusion limited to the measured boundary.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P38.2-A. Durations are 2,3,3,4,8,10 ms after warmup. Compute mean, median and nearest-rank p95.

P38.2-B. Variant B is 20% faster but skips one result in every 1,024. May its timings be accepted for an exact operation?

Transfer task

Create two benchmark protocols: one for interactive p95 latency and one for sustained batch throughput. Explain why their collection windows, summaries and conclusions differ.

Assigned study: R26: profiling; R37: performance analysis; R09: inference and uncertainty.

Lab 38.3 — SIMD, cache blocking, NUMA and job scheduling

Original practical requirement

Optimize a particle or transform workload using one locality change and one scheduling change. Pass: test tail sizes, report numerical differences, and measure small as well as large workloads to expose overhead.

Automatically checked slice

Model job placement. costs are nonnegative service times, workers is 1..256, overhead is per-job service cost. policy is round_robin or greedy (earliest available worker, ties lowest index). All jobs are ready at t=0. This does not simulate NUMA, work stealing, real threads or vector instructions.

Starter: `labs/starters/lesson_38_3.py`

Demo: `python -m labs.run demo 38.3`

Student check: `python -m labs.run check 38.3 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "costs": [8, 1, 8, 1],
  "workers": 2,
  "overhead": 0,
  "policy": "round_robin"
}
```

Full lab procedure

1. Create a verified sequential particle or transform baseline and include zero, one, tail, medium and large counts.
2. Implement the scheduling slice, tracing each assignment and distinguishing ready jobs from dependency-blocked jobs.
3. Apply one locality change such as field layout or tiling, and one scheduling change such as coarser jobs or load-aware placement. Keep their experiments separable.
4. Report timings, useful work, numerical differences and memory costs. Inspect compiler output or reports before claiming SIMD execution.

Evidence and acceptance

Submit baseline and variants, a correctness suite with tails, a placement timeline, raw measurements at multiple scales, numerical-error results and a reasoned crossover/fallback policy.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P38.3-A. Sixty-four jobs each perform 2 microseconds of work and pay 3 microseconds overhead. On one worker, what fraction of total time is overhead?

P38.3-B. In a hypothetical machine with 64-byte coherence lines, two 8-byte counters begin at offsets 0 and 8 of the same line. Is updating them independently sufficient to avoid false sharing?

Transfer task

Add a dependency that prevents the second 8-unit job from starting until the first completes. Recompute the lower bound and explain why load balancing cannot remove the dependency.

Assigned study: R37: parallel memory patterns; R48: multiprocessors and memory systems; R11: safe C++ and concurrency.

Lab 38.4 — Energy, thermal behavior and sustained performance

Original practical requirement

Run a sustained workload within normal device limits or analyze a synthetic power trace. Pass: distinguish watts from joules, report the measurement boundary and compare early versus sustained performance.

Automatically checked slice

Integrate interval-average power. times_s has one more item than power_w and completed_frames, and is strictly increasing. Each power value describes the entire following interval, not an instantaneous sample. Return energy, duration, average power and completed frames per joule; zero energy yields null efficiency.

Starter: `labs/starters/lesson_38_4.py`

Demo: `python -m labs.run demo 38.4`

Student check: `python -m labs.run check 38.4 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "times_s": [0, 2],
  "power_w": [50],
  "completed_frames": [120]
}
```

Full lab procedure

1. Reproduce the two original energy examples and write the measurement boundary beside each.
2. Analyze data/synthetic_power_trace.json with the interval-average convention. Compare early and sustained windows and retain the provenance record.
3. Implement the calculator, test nonuniform intervals and reject invalid durations or negative powers. Explain how zero energy is represented.
4. Design a sustained normal-limits experiment or analyze the supplied synthetic case; specify a quality-preserving tier policy with hysteresis.

Evidence and acceptance

Submit the trace with provenance, integration worksheet, early/sustained comparisons, useful-quality definition, normal-limits protocol and a clearly labeled uncertainty/limitations section.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P38.4-A. Three interval-average samples are 20 W for 2 s, 30 W for 1 s and 10 W for 3 s. Find energy and average power over the full interval.

P38.4-B. A system produces 1,200 accepted frames using 800 J. Another produces 1,000 using 500 J at the same quality. Compare frames per joule.

Transfer task

Include 20 seconds of post-task idle energy in the 50 W and 80 W example under two different idle-power assumptions. Identify which missing assumption could reverse the conclusion.

Assigned study: R48: quantitative architecture; R51: device power management; R37: GPU execution and performance context.

Module 38 — Project and review

A reproducible performance investigation

Use the expanded project plan at this module's opening. Bring its implementation and evidence to the review below.

Project review rule

Assess contract/requirements, implementation, verification, evidence/reproducibility and limitations at 0–4 points each. A suggested readiness target is 16/20 after revision, with no unresolved correctness, ownership or safety defect. This local study rubric is not an accredited assessment. Gate H has its own integrated rubric.

Five-question module review

R38-1. A kernel-only speedup is 8x but the kernel occupied 5% of a frame. Find the no-overhead additive frame speedup.

R38-2. Your fastest implementation fails only at N=17. How should performance results be handled?

R38-3. A parallel version wins at N=1,000,000 but loses at N=100. What should the report recommend?

R38-4. A power trace uses endpoint instantaneous samples but the script treats them as interval averages. What must change?

R38-5. A counter correlates with frame spikes. What additional work supports causality?

Have a reviewer select one adverse case and rerun it from your instructions. Record the reviewer's concern, evidence examined and correction or reasoned response.

Module 39 — Practical work

Prerequisites: Modules 22, 27, 29 and 36-38.

Original project: Create a platform capability and latency report for the engine. Implement one non-desktop constraint in simulation or on an available device: touch input, suspend/resume, restricted memory, variable presentation or XR timing.

Expanded project plan

Create a strict engine backend simulation that enforces a memory budget, read-only packaged assets, supported save behavior and at least one lifecycle transition. Connect capability negotiation to behavioral failures rather than just checking feature names. Implement a quality tier whose changes do not alter authoritative simulation rules.

Trace input to a named software presentation marker and report interval pacing separately from latency. Mark unmeasured display stages explicitly. Build a two-view numerical camera exercise with declared space/time conventions and tracking-loss behavior; a headset is not required.

Test transition peak memory, sudden kill before a checkpoint, resource-generation mismatch, unsupported optional service and permission-denied save. Deliver the capability matrix, trace, memory/lifecycle tests, camera tests and a report that separates simulation from device measurements and makes no proprietary-console or optical-latency claim without evidence.

Lab 39.1 — Input-to-display latency and frame pacing

Original practical requirement

Instrument input receipt, simulation consumption, render submission and presentation-related timestamps. Pass: explain which stages are actually measured and which remain estimated, and show a frame-pacing distribution.

Automatically checked slice

Summarize a synthetic one-clock input/presentation trace. input_ms and present_ms are same-length lists with at least two rows; presentation is strictly increasing and never precedes its corresponding input. timestamp_kind must equal synthetic_present. Return mean/p95 input-to-modeled-present and presentation-interval extrema. No photon measurement is inferred.

Starter: `labs/starters/lesson_39_1.py`

Demo: `python -m labs.run demo 39.1`

Student check: `python -m labs.run check 39.1 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "input_ms": [0, 10, 20],
  "present_ms": [20, 30, 40],
  "timestamp_kind": "synthetic_present"
}
```

Full lab procedure

1. Draw a complete input-to-display timeline and name each clock and event boundary. Mark unknown physical stages.
2. Implement the synthetic trace analyzer and validate monotonically ordered presentations and nonnegative input-to-marker delays.
3. Analyze data/synthetic_latency_trace.json, comparing typical and stalled intervals without dropping inconvenient frames.
4. Change one modeled queue policy or instrument an accessible engine; compare latency distribution, interval distribution and useful frame throughput together.

Evidence and acceptance

Submit the annotated timeline, raw trace with boundary definitions, mean and p95 input-to-marker latency, interval distribution, and a statement separating measured, modeled and unknown display stages.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P39.1-A. Input occurs at 8 ms and the software presentation marker at 42 ms. The panel response is unmeasured. What latency may be reported?

P39.1-B. Presentation intervals are 10,10,40 ms. Find total-throughput FPS and compare it with the arithmetic mean of interval FPS.

Transfer task

Create two synthetic traces with identical average FPS but different queue ages and interval tails. Explain which measurements distinguish them and which do not.

Assigned study: R22: input and event timing; R24 or R25: presentation; R52: frame timing as an advanced comparison.

Lab 39.2 — Mobile and handheld systems: memory, power and lifecycle

Original practical requirement

Add suspend/resume simulation and a constrained memory mode. Pass: progress survives the documented lifecycle, resources are restored safely and quality changes do not alter authoritative simulation rules.

Automatically checked slice

Simulate persistent progress independently of volatile graphics resources. Start RUNNING at tick 0, resource generation 1. Events: step, checkpoint, suspend, resume, kill, launch. suspend checkpoints in this educational contract; sudden kill does not. kill discards uncheckpointed progress. Resuming rebuilds resources with a new generation. Unsupported transitions raise ValueError.

Starter: `labs/starters/lesson_39_2.py`

Demo: `python -m labs.run demo 39.2`

Student check: `python -m labs.run check 39.2 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "events": ["step", "step"]
}
```

Full lab procedure

1. Separate durable progress, reconstructible resources and transient state in a table. Define a checkpoint loss budget.
2. Implement the supplied lifecycle state machine and test unexpected kill, illegal work while suspended and repeated recreation.
3. Add a strict memory budget to a small backend or simulation; include overlapping old/new resources during quality changes.
4. Exercise checkpoint corruption or interrupted persistence in a disposable prototype and show a defined recovery path without invalid handles.

Evidence and acceptance

Submit lifecycle and persistence contracts, peak-memory arithmetic, state-transition traces, failure-injection results and a quality policy that preserves authoritative gameplay.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P39.2-A. A budget is 512 MiB. Persistent state is 260 MiB; current and replacement resource sets each need 140 MiB. Can both coexist?

P39.2-B. A save claims version 9 but the loader supports versions 1–3. What is a defensible response?

Transfer task

Implement a bounded allocator for your model and make a quality transition fail halfway through. Demonstrate rollback or recovery without leaking the old resource set.

Assigned study: R25: platform graphics resources; R22: application lifecycle events; R29: text and input usability; R51: power-management concepts.

Lab 39.3 — XR spaces, stereo views, pose timing and comfort constraints

Original practical requirement

Implement two simulated camera views and a pose-time log, or a minimal OpenXR scene on available supported hardware. Pass: spaces and units are explicit, frame timing follows the runtime and loss of tracking has defined behavior.

Automatically checked slice

Compute an educational stereo pose in meters with +Y up and -Z forward. Head yaw is right-handed about +Y. Constant linear velocity predicts head translation between sample_s and target_s (0..0.1 seconds ahead). Eye offsets +/-ipd_m/2 are rotated into world coordinates. tracking_valid=False returns no invented pose. This is not an OpenXR runtime implementation.

Starter: `labs/starters/lesson_39_3.py`

Demo: `python -m labs.run demo 39.3`

Student check: `python -m labs.run check 39.3 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "tracking_valid": true,
  "position_m": [0, 0, 0],
  "velocity_mps": [0, 0, 0],
  "ipd_m": 0.064,
  "sample_s": 0,
  "target_s": 0,
  "yaw_deg": 0
}
```

Full lab procedure

1. Declare coordinate axes, units, yaw sign, tracking space and target time in a contract.
2. Implement the stereo prediction slice, including zero and 90-degree yaw and explicit tracking loss.
3. Build two simulated camera views and test their inverse transforms on known world points. Keep runtime-specific projection assumptions separate.
4. Inject an origin change, stale pose and invalid tracking; document recovery and the limits of the constant-velocity model.

Evidence and acceptance

Submit coordinate and timing contracts, stereo/view-transform tests, prediction-error analysis under a changed-motion case, and tracking-loss traces with no invented valid pose.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P39.3-A. Constant speed is 0.5 m/s and prediction horizon is 30 ms. Find predicted displacement.

P39.3-B. A camera at world position (2,0,0) has identity rotation. Where does world point (2,0,-3) appear in camera coordinates?

Transfer task

Add a constant-acceleration truth trajectory and compare it with the constant-velocity predictor. Plot or tabulate error against horizon without claiming this validates a real headset predictor.

Assigned study: R52: spaces, view configuration, frame loop and timing; R29: interaction and accessibility review.

Lab 39.4 — Portability, platform services and console-development boundaries

Original practical requirement

Write a platform interface and capability matrix, then implement a second backend or strict mock. Pass: assumptions about paths, input and lifecycle are exposed, and proprietary requirements are marked unavailable until authorized.

Automatically checked slice

Negotiate required and optional capabilities without guessing vendor access. supported, required and optional are lists of strings. Return sorted missing requirements and enabled optional capabilities. A backend is accepted only if every required capability is advertised. This structural check does not certify a platform.

Starter: `labs/starters/lesson_39_4.py`

Demo: `python -m labs.run demo 39.4`

Student check: `python -m labs.run check 39.4 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "supported": ["keyboard", "save", "ray"],
  "required": ["keyboard", "save"],
  "optional": ["ray", "touch"]
}
```

Full lab procedure

1. Write required and optional capability sets and define each missing-capability outcome.
2. Implement the negotiator and test unsupported requirements, duplicate/invalid capability data and optional omissions.
3. Build a strict backend mock with read-only assets, writable-save constraints and an injected unavailable service.
4. Run an engine slice through the mock and report which desktop assumptions were exposed and how the implementation responds.

Evidence and acceptance

Submit the capability matrix, strict mock, permission and lifecycle failure traces, save-path contract and a clear statement that no proprietary platform access or certification is claimed.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P39.4-A. Required features are render/save; available features are render/haptics. Is startup allowed by the contract?

P39.4-B. A mock returns success for a save to a read-only packaged-asset path. What test is missing?

Transfer task

Revoke an optional service after initialization while a request is pending. Specify callback ownership, cancellation and user-visible behavior, then make the mock enforce the specification.

Assigned study: R22, R24 and R25: platform interfaces; R29: cross-device usability; official authorized platform documentation only for restricted targets.

Module 39 — Project and review

A constrained-platform and latency report

Use the expanded project plan at this module's opening. Bring its implementation and evidence to the review below.

Project review rule

Assess contract/requirements, implementation, verification, evidence/reproducibility and limitations at 0–4 points each. A suggested readiness target is 16/20 after revision, with no unresolved correctness, ownership or safety defect. This local study rubric is not an accredited assessment. Gate H has its own integrated rubric.

Five-question module review

R39-1. Two engines average 60 FPS; one reports much older input at presentation. Explain a likely distinction.

R39-2. A quality change fits steady memory but fails during recreation. What budget was omitted?

R39-3. Stereo looks correct at zero yaw but eye positions are wrong after a turn. Identify a likely transform bug.

R39-4. A save service disappears after capability negotiation. What should a robust engine do?

R39-5. List the boundaries a platform report must avoid overstating.

Have a reviewer select one adverse case and rerun it from your instructions. Record the reviewer's concern, evidence examined and correction or reasoned response.

Module 40 — Practical work

Prerequisites: Gates F and G; Modules 36-39.

Original project: Gate H: choose one game workload and compare a software baseline with a specialized hardware or architecture alternative. Deliver a reference implementation, model or RTL, software interface, verification evidence and an end-to-end tradeoff report.

Expanded project plan

Integrate the preceding projects into one workload-centered comparison. Do not submit five unrelated demos as a capstone. Use the full CAPSTONE_SPEC to choose a measured engine operation, define quality and latency requirements, defend a competent baseline, compare at least two architecture alternatives and implement one bounded specialized path.

The provided affine pipeline is a teaching reference. Adapt its numerical contract, host/device stride, workload distribution and failure handling to your own engine use. Label assumed clocks, rates and resource counts; measure the software baseline and keep the timing boundaries comparable. Evaluate small, large and adverse cases and show what changes when memory bandwidth, capacity or latency is tightened.

Deliver a reproducible portfolio, raw data, source, tests, an evidence ledger and an independently reviewed recommendation. A conclusion favoring software is acceptable; a conclusion supported only by optimistic model parameters is not. Gate H remains incomplete until the learner's project evidence and review are supplied.

Lab 40.1 — Workload characterization and measurable requirements

Original practical requirement

Select a workload from the engine and write a requirement sheet plus baseline measurements. Pass: each target has units and a test, and the maximum plausible application benefit is estimated.

Automatically checked slice

Calculate a bounded additive-frame proposal. frame_ms >0; 0<= operation_ms <= frame_ms; acceleration >0; overhead_ms >=0. maximum_saved_fraction assumes the operation disappears with zero overhead. New frame time uses the supplied finite acceleration. An overlapped operation off the critical path cannot use this model without justification.

Starter: `labs/starters/lesson_40_1.py`

Demo: `python -m labs.run demo 40.1`

Student check: `python -m labs.run check 40.1 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "frame_ms": 20,
  "operation_ms": 0.2,
  "acceleration": 2,
  "overhead_ms": 0
}
```

Full lab procedure

1. Capture a real engine operation or an equivalent measured workload with justified relevance; record small, typical, large and adverse cases.
2. Write a requirements table and compute the maximum application benefit before choosing hardware.
3. Establish and validate a competent software baseline with raw measurements and the same numerical-quality contract as all alternatives.
4. Create requirement-to-test and claim-to-evidence links, including explicit conditions that would reject the proposal.

Evidence and acceptance

Submit the workload dossier, measured baseline, maximum-benefit worksheet, numerical-quality contract, requirements traceability and a justified decision about whether specialization deserves further work.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P40.1-A. An operation takes 3 ms in a 15 ms frame. It improves 3x but adds 0.4 ms setup/transfer. Find new frame time and speedup.

P40.1-B. A subsystem consumes 4 ms of CPU time but fully overlaps an independent 7 ms GPU path. Does removing all 4 ms necessarily save 4 ms of frame time?

Transfer task

Write a one-page rejection memo for a tempting accelerator whose maximum application benefit is smaller than the uncertainty in your current measurement. Propose the next measurement, not a fabricated conclusion.

Assigned study: R43: bottleneck modeling; R48: quantitative design; R49: project methodology.

Lab 40.2 — Architecture alternatives, arithmetic precision and dataflow

Original practical requirement

Model three variants, such as scalar, pipelined and multi-lane. Pass: resource and bandwidth estimates are explicit, fixed-point error meets a chosen application tolerance and rejected variants have documented reasons.

Automatically checked slice

Apply a signed fixed-point affine transform with explicit rounding and saturation. x and gain are signed encoded integers with bits (2..32) and frac (0..bits-1) fractional bits. Compute nearest-even(x*gain / 2**frac) + bias, then saturate to bits. bias is an encoded signed integer in the same format. Return raw integer outputs; Python arbitrary-precision intermediates avoid overflow.

Starter: `labs/starters/lesson_40_2.py`

Demo: `python -m labs.run demo 40.2`

Student check: `python -m labs.run check 40.2 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "bits": 16,
  "frac": 8,
  "gain": 256,
  "bias": 0,
  "x": [-256, 0, 256, 32767]
}
```

Full lab procedure

1. Specify exact arithmetic, units, range, rounding, saturation and an independently checkable oracle.
2. Implement the fixed-point slice and test ties, negative values, saturation and invalid formats.
3. Draw at least two architecture alternatives and count required transfers at each declared memory level. Compare compute and bandwidth ceilings.
4. Run or adapt the bounded pipeline model; verify tails and backpressure, then sweep lane count and memory bandwidth without relabeling assumptions as measurements.

Evidence and acceptance

Submit the arithmetic contract, boundary tests, two alternative dataflow diagrams, cycle convention, model traces, bandwidth sensitivity and explicitly tiered resource/timing estimates.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P40.2-A. Eight groups pass through an initiation-interval-one model with latency four cycles and no stalls. What is first-acceptance to last-retirement elapsed time?

P40.2-B. An operation moves 8 bytes/item at a declared 240 MB/s. What is its bandwidth-limited item rate, ignoring other costs?

Transfer task

Change the rounding rule to truncation toward zero and find inputs where it disagrees with nearest-even. Quantify the bias over a declared distribution rather than only displaying one favorable example.

Assigned study: R49: architecture and RTL modeling; R43: bandwidth limits; R33: resource inference.

Lab 40.3 — Interface design, virtualization and end-to-end evaluation

Original practical requirement

Implement a host driver model with invalid-descriptor, timeout and reset tests. Pass: buffer bounds are enforced, ownership survives failure and results include both kernel and end-to-end timing.

Automatically checked slice

Simulate one registered 4-byte-word buffer with explicit ownership. capacity is bytes. Events submit(offset,count), timeout, complete(epoch), reset. Submission validates alignment and subtraction-based bounds; zero count is rejected. A timeout quarantines the buffer and completion cannot release a quarantined buffer. reset models acknowledged device quiescence, advances epoch, and reclaims the buffer. No actual DMA occurs.

Starter: `labs/starters/lesson_40_3.py`

Demo: `python -m labs.run demo 40.3`

Student check: `python -m labs.run check 40.3 --student`

Published contract cases: 8. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "capacity": 16,
  "events": [
    {
      "op": "submit",
      "offset": 0,
      "count": 4
    },
    {
      "op": "complete",
      "epoch": 0
    }
  ]
}
```

Full lab procedure

1. Write a descriptor schema with actual stride, range, alignment, operation, format, request identity and ownership rules.
2. Implement the supplied ownership slice and test invalid descriptors, stale/duplicate completion, timeout quarantine and acknowledged-quiescent reset.
3. Connect an adapted contract to your chosen capstone architecture model, documenting every mismatch between standalone teaching examples and the integrated design.

4. Compare end-to-end baseline and alternatives across real batch distributions, including adverse completion delay and a fallback path.

Evidence and acceptance

Submit the interface specification, state machine, descriptor rejection tests, ownership/failure traces, end-to-end phase budget and an explicit statement of any unimplemented isolation mechanism.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P40.3-A. A 1,024-byte buffer uses 4-byte elements. Offset is 1,000 and count is 7. Is the request valid?

P40.3-B. CPU time is 0.3 microseconds/item; an alternative costs 15 microseconds setup plus 0.1 microseconds/item. Find the first integer batch size that wins.

Transfer task

Add two registered contexts to the descriptor model and construct a request that is in bounds numerically but belongs to the wrong context. Require rejection before any modeled access.

Assigned study: R51: DMA and device interfaces; R49: system integration; R38: modeled systems when hardware is unavailable.

Lab 40.4 — Design review, evidence tiers and undergraduate CE completion

Original practical requirement

Complete Gate H and a design-review presentation. Pass: every major claim traces to an artifact, the baseline is reproduced and a limitations section clearly separates educational estimates from production claims.

Automatically checked slice

Check evidence-ledger structure. Each claim needs id, tier, artifact, requirement, limitations; known_artifacts is a list of paths. Allowed tiers: analytical, functional_model, cycle_model, synthesis, host_measurement, physical_measurement. Report structural omissions, not scientific approval. Arbitrary unsupported tiers never become measurements by renaming files.

Starter: `labs/starters/lesson_40_4.py`

Demo: `python -m labs.run demo 40.4`

Student check: `python -m labs.run check 40.4 --student`

Published contract cases: 6. Add at least one unseen boundary and one adverse case justified by your contract.

Example input (predict the output before running it):

```
{
  "claims": [
    {
      "id": "C1",
      "tier": "analytical",
      "artifact": "model.json",
      "requirement": "R1",
      "limitations": "Assumed clock; no placement or routing."
    }
  ],
  "known_artifacts": ["model.json"]
}
```

Full lab procedure

1. Create a claim ledger assigning an evidence tier and source artifact to every major correctness, timing, memory and resource assertion.
2. Run the ledger checker and inspect every referenced artifact; explain which semantic review tasks the checker cannot perform.
3. Complete the Gate H portfolio and have another person reproduce the core commands and challenge the baseline and adverse cases.
4. Revise unsupported claims, record remaining limitations and present a recommendation that addresses both application benefit and tighter memory/latency constraints.

Evidence and acceptance

Submit the reproducible portfolio, evidence ledger, review responses, full comparison including negative cases, and a conclusion that distinguishes learning completion from professional or fabrication authorization.

Keep raw artifacts, a short explanation of limitations and the exact reproduction command. Report whether each result is analytical, modeled or measured. The checker assesses the published slice contract, not the scientific sufficiency of the full assignment.

Written problems

P40.4-A. A cycle model predicts 12 microseconds at an assumed clock. A host benchmark measures 18 microseconds for a different implementation. Can the report claim a measured 1.5x hardware speedup?

P40.4-B. The main dataset changes after hashes are recorded. What should the evidence workflow do?

Transfer task

Conduct a red-team review of your own strongest performance claim. Find the most plausible alternative explanation, design a discriminating test and record whether the claim survives.

Assigned study: R49: design-project reporting; R48: architectural tradeoffs; revisit the evidence and project rubrics in the handbook.

Module 40 — Project and review

Gate H: a defensible specialization decision

Use the expanded project plan at this module's opening. Bring its implementation and evidence to the review below.

Project review rule

Assess contract/requirements, implementation, verification, evidence/reproducibility and limitations at 0–4 points each. A suggested readiness target is 16/20 after revision, with no unresolved correctness, ownership or safety defect. This local study rubric is not an accredited assessment. Gate H has its own integrated rubric.

Five-question module review

R40-1. What minimum alternatives and evidence are required by Gate H?

R40-2. Four lanes produce no gain over one in the supplied model. Explain the asymptotic limit.

R40-3. A timed-out request resets its software epoch, but old hardware is still writing. Is isolation restored?

R40-4. A model's assumed 100 MHz clock is accidentally labeled measured. How should the review respond?

R40-5. An accelerator loses to improved software on every representative batch. Can the capstone pass?

Have a reviewer select one adverse case and rerun it from your instructions. Record the reviewer's concern, evidence examined and correction or reasoned response.

Reading directory

Resolve all R- and S-codes in SOURCES.md, STUDENT_HANDBOOK or COURSE.html. The source directory includes official resources and the original assigned-reading metadata; external reading is not bundled.